



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

Semester 2

Session 2025/2026

SECJ3483 - SECTION 01
WEB TECHNOLOGY

Group Lab Activity: Security Investigation and Fix

Project Title : Secure Person BMI Full-Stack Web Application

Name	Matrics No.
MUHAMMAD AIDIL HAIKAL BIN MAZALAN	A23CS5010
MUHAMMAD AIMAN DANISH BIN MUHAMMAD EKHSAN	A23CS0119
MUHAMMAD FAKHRUL RAZZI BIN MD NOOR	A23CS0128
MUHAMMAD RIFQI BIN RAZALI	A23CS0136

2. Security investigation summary

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Data is accepted and stored in the database.	Yes	Missing backend validation because frontend validation can be bypassed using ThunderClient.
2	Access another user's BMI record	Backend should deny access	The private BMI record data is returned.	Yes	Broken Object Level Authorization because data ownership is not verified after a user logs in.
3	Access staff route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because there are no server-side role checks for staff features.
4	Access admin route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because the route lacks server-side admin protection middleware.
5	Submit XSS payload in notes	Payload should display as text	The browser executes the script by showing the alert.	Yes	Stored Cross-Site Scripting because the frontend renders input unsafely using v-html.
6	Check API response	Password/hash should not be visible	The password value is visible in the payload.	Yes	Sensitive data exposure because the API blindly returns all columns including password hash.
7	Try SQL Injection input	Input should be treated as data	The structural database query behaves unexpectedly.	Yes	SQL Injection because the application constructs database queries using unsafe string concatenation.
8	Try to update protected fields	Backend should reject or ignore protected fields	The fields (e.g., role, user_id) are accepted and modified	Yes	Mass assignment because the backend blindly executes updates on protected fields like user_id and role.

Test 1: Add BMI with invalid data

- Did the backend accept the data?

Yes, it saved the negative and empty values directly into the database.

- Did the frontend validation stop you?

No, because I can bypass the frontend UI using Thunder Client.

- Can the same request be sent using Postman?

Yes, anyone can intercept and send raw requests to the API.

- Where should the validation be enforced?

Backend, because frontend validation can be bypassed.

The screenshot shows the Thunder Client interface with a POST request to `http://localhost:8080/api/persons`. The request body is a JSON object with invalid data: `{ "name": "", "age": -5, "height": 0, "weight": -70, "notes": "testing invalid BMI" }`. The response status is `201 Created` with a size of `691 Bytes` and a time of `44 ms`. The response body is a JSON object containing a message, a person object, and debug information.

```
1 {
2   "message": "BMI record created. This route trusts frontend data.",
3   "person": {
4     "id": 22,
5     "user_id": 1,
6     "name": "",
7     "age": -5,
8     "height": "0.00",
9     "weight": "-70.00",
10    "bmi": "0.00",
11    "category": "",
12    "notes": "testing invalid BMI",
13    "created_at": "2026-06-26 11:08:22"
14  },
15  "debug_received_body": {
16    "name": "",
17    "age": -5,
18    "height": 0,
19    "weight": -70,
20    "notes": "testing invalid BMI"
21  },
22  "debug_sql": "INSERT INTO persons (user_id, name, age, height,
23               weight, bmi, category, notes)\n                VALUES (1, '',
24               -5, 0, -70, 0, '', 'testing invalid BMI')"
```

Test 2: Access another user's BMI record

- Did the backend return another user's BMI record?

Yes, changing the ID in the URL leaked other people's data.

- Is login alone enough?

No, it is just authentication but no authorization.

- What should the backend compare?

It must match the user_id from the JWT with the user_id on the database.

- Should staff and admin have different access?

Yes, staff and admin roles need to have different access to monitor all records.

Thunder Client interface showing an HTTP GET request to `http://localhost:8080/api/persons/1`. The request headers include `Accept: */*`, `User-Agent: Thunder Client (https://www.thunderclient.com)`, and `Authorization: eyJ1c2VyX2lkjoxLCljb2xlljoidXNlcislmVtY`. The response status is 200 OK, with a size of 435 Bytes and a time of 46 ms. The response body is a JSON object:

```
1 {
2   "message": "Record returned without ownership check.",
3   "person": {
4     "id": 1,
5     "user_id": 1,
6     "name": "Muhammad Aiman Hakimi",
7     "age": 21,
8     "height": "1.70",
9     "weight": "65.00",
10    "bmi": "22.49",
11    "category": "Normal",
12    "notes": "Student sample BMI record",
13    "created_at": "2026-06-22 11:28:30"
14  },
15  "debug_sql": "SELECT * FROM persons WHERE id = 1"
16 }
```

Thunder Client interface showing an HTTP GET request to `http://localhost:8080/api/persons/3`. The request headers include `Accept: */*`, `User-Agent: Thunder Client (https://www.thunderclient.com)`, and `Authorization: eyJ1c2VyX2lkjoxLCljb2xlljoidXNlcislmVtY`. The response status is 200 OK, with a size of 435 Bytes and a time of 50 ms. The response body is a JSON object:

```
1 {
2   "message": "Record returned without ownership check.",
3   "person": {
4     "id": 3,
5     "user_id": 3,
6     "name": "Ahmad Farhan Roslan",
7     "age": 21,
8     "height": "1.75",
9     "weight": "82.00",
10    "bmi": "26.78",
11    "category": "Overweight",
12    "notes": "Needs weight monitoring",
13    "created_at": "2026-06-22 11:28:30"
14  },
15  "debug_sql": "SELECT * FROM persons WHERE id = 3"
16 }
```

Test 3: Access staff route as normal user

- Did the backend block the request?

No, a normal user token could access the restricted endpoints.

- Which roles should be allowed?

Only users with a role value of 'staff' or 'admin'.

- Should checking be done in frontend route guard only?

No, attackers don't use the frontend browser to make API requests.

- Where must the final decision happen?

In the backend routing middleware.

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/staff/persons` and its response. The response status is 200 OK, with a size of 9.7 KB and a time of 22 ms.

Request Details:

- Method: GET
- URL: `http://localhost:8080/api/staff/persons`
- Headers:
 - Accept: */*
 - User-Agent: Thunder Client (https://www.thunderclient.com)
 - Authorization: `eyJ1c2VyX2lkjoxLCjyb2xljoidXNlcislmVtY`
 - Content-Type: application/json
 - header: value

Response Details:

```
1 {
2   "message": "All BMI records returned without staff role check."
3 }
4 {
5   "id": 22,
6   "user_id": 1,
7   "name": "",
8   "age": -5,
9   "height": "0.00",
10  "weight": "-70.00",
11  "bmi": "0.00",
12  "category": "",
13  "notes": "testing invalid BMI",
14  "created_at": "2026-06-26 11:08:22",
15  "owner_email": "aiman@student.utm.my",
16  "owner_role": "user"
17 },
18 {
19   "id": 21,
20   "user_id": 1,
21   "name": "",
22   "age": -5,
23   "height": "0.00",
24   "weight": "-70.00",
25   "bmi": "999.00",
26   "category": "Normal",
27   "notes": "invalid test",
28   "created_at": "2026-06-26 10:42:47",
29   "owner_email": "aiman@student.utm.my",
30   "owner_role": "user"
31 },
32 {
33   "id": 20,
34   "user_id": 20,
35   "name": "Mazlina Ahmad",
36   "age": 45,
37   "height": "1.60",
38   "weight": "58.00"
```

Test 4: Access admin route as normal user

- Did the backend accept user_id?

Yes.

- Did the backend accept role?

Yes.

- Did the backend accept bmi and category from the frontend?

Yes, the database blindly updated them all.

- Which fields should be allowed?

Only name, age, height, weight, and notes.

- Which fields should be calculated or controlled by backend?

Structural fields like user_id and role, plus calculations like bmi and category.

The screenshot shows a web client interface with a request tab selected. The request is a GET to `http://localhost:8080/api/admin/users`. The response is a 200 OK status with a 5.8 KB size and 26 ms time. The response body is a JSON object with a message and a list of users.

Request:

```
GET http://localhost:8080/api/admin/users
```

Response:

```
{
  "message": "All users returned without admin role check. Sensitive fields exposed.",
  "users": [
    {
      "id": 1,
      "name": "Muhammad Aiman Hakimi",
      "email": "aiman@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 2,
      "name": "Nur Aisyah Zulkifli",
      "email": "aisyah@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 3,
      "name": "Ahmad Farhan Roslan",
      "email": "farhan@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 4,
      "name": "Siti Nur Balqis Ismail",
      "email": "balqis@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    }
  ]
}
```

Test 5: Submit XSS payload in notes

- Does the API return password_hash?
Yes, it was fully visible in the JSON response payload.
- Does the frontend really need this data?
No, the client UI never needs to know password hashes.
- What fields should be returned?
Only safe profile data: id, name, email, and role.
- How can SQL SELECT be improved?
Stop using SELECT * and explicitly write out the safe columns.

The screenshot shows a REST client interface in VS Code. The request is a POST to `http://localhost:8080/api/persons`. The response is a 201 Created status with a 769 Byte body. The JSON response contains a message, a person object, and debug information. The person object includes fields like id, user_id, name, age, height, weight, bmi, category, notes, and created_at. The notes field contains an XSS payload: `<img src=x onerror=alert('XSS')\>`. The debug information shows the received body and the SQL query used to insert the record.

```
POST http://localhost:8080/api/persons

JSON Content
1 {
2   "name": "XSS Test",
3   "age": 25,
4   "height": 1.75,
5   "weight": 70,
6   "notes": "<img src=x onerror=alert('XSS')\>"
7 }

Response
1 {
2   "message": "BMI record created. This route trusts frontend data.",
3   "person": {
4     "id": 23,
5     "user_id": 1,
6     "name": "XSS Test",
7     "age": 25,
8     "height": "1.75",
9     "weight": "70.00",
10    "bmi": "0.00",
11    "category": "",
12    "notes": "<img src=x onerror=alert('XSS')\>",
13    "created_at": "2026-06-26 11:38:19"
14  },
15  "debug_received_body": {
16    "name": "XSS Test",
17    "age": 25,
18    "height": 1.75,
19    "weight": 70,
20    "notes": "<img src=x onerror=alert('XSS')\>"
21  },
22  "debug_sql": "INSERT INTO persons (user_id, name, age, height, weight, bmi, category, notes)\nVALUES (1, 'XSS Test', 25, 1.75, 70, 0, '', '<img src=x onerror=alert('XSS')\>')\n"
23 }
```

The screenshot shows the 'Person BMI Security Lab' web application. The 'My BMI Records' section displays a message: 'Investigation hint: Does the backend return only your records? What happens if you request another ID?'. Below this, there are buttons for 'Load My Records' and 'Try Manual ID'. A green message box states: 'BMI records returned. This route is intentionally weak.' The 'Raw API response' section shows the JSON response from the API, which includes the XSS payload in the notes field: `<img src=x onerror=alert('XSS')\>`.

Test 6: Check API response

- Did the alert execute?

Yes, the script payload popped up in the browser window.

- Is the frontend using innerHTML or v-html?

Yes, it used unsafe binding instead of text escaping.

- What is safer for displaying normal text?

Double curly braces {{ }} text interpolation or textContent.

- Should user-generated content be rendered as HTML?

No, never render raw user input as live HTML.

The screenshot shows the Thunder Client interface. The top bar displays the current request: GET http://localhost:8080/api/admin/users. The left sidebar shows the 'Headers' tab selected, with a table of HTTP headers:

Header	Value
Accept	*/
User-Agent	Thunder Client (https://www.thunderclient.)
Authorization	eyJ1c2VyX2lkjoxLCJyb2xljoidXNlcilslmVtY
Content-Type	application/json
header	value

The right pane shows the 'Response' tab, displaying the JSON response from the server. The status is 200 OK, size is 5.8 KB, and time is 39 ms. The response body is a JSON object with a message and an array of user data:

```
1 {
2   "message": "All users returned without admin role check. Sensitive fields exposed.",
3   "users": [
4     {
5       "id": 1,
6       "name": "Muhammad Aiman Hakimi",
7       "email": "aiman@student.utm.my",
8       "password": "password123",
9       "password_hash": "password123",
10      "role": "user",
11      "created_at": "2026-06-22 11:28:30"
12    },
13    {
14      "id": 2,
15      "name": "Nur Aisyah Zulkifli",
16      "email": "aisyah@student.utm.my",
17      "password": "password123",
18      "password_hash": "password123",
19      "role": "user",
20      "created_at": "2026-06-22 11:28:30"
21    },
22    {
23      "id": 3,
24      "name": "Ahmad Farhan Roslan",
25      "email": "farhan@student.utm.my",
26      "password": "password123",
27      "password_hash": "password123",
28      "role": "user",
29      "created_at": "2026-06-22 11:28:30"
30    },
31    {
32      "id": 4,
33      "name": "Siti Nur Balqis Ismail",
34      "email": "balqis@student.utm.my",
35      "password": "password123",
36      "password_hash": "password123",
37      "role": "user",
38      "created_at": "2026-06-22 11:28:30"
39    }
40  ]
41 }
```


Test 7: Try SQL Injection input

- Did the SQL query behave unexpectedly?

Yes, inputting quotes broke out of the string and changed the logic.

- Was login bypassed?

Yes, the payload successfully manipulated the query parameters.

- Was extra data returned?

Yes, it fetched records without validating real credentials.

- Is the SQL query using string concatenation?

Yes, it directly glued variables into the raw query string.

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/login`. The request body is a JSON object with the following content:

```
1 {
2   "email": "aiman@student.utm.my' -- ",
3   "password": "blablabla"
4 }
```

The response status is **200 OK** with a size of **735 Bytes** and a time of **49 ms**. The response body is a JSON object with the following content:

```
1 {
2   "message": "Login successful. This token is intentionally insecure",
3   "token": "eyJ1c2VyX2lkIjoxLCJyb2x1IjoiaXNlcjIsImVtYWlsIjoiaWltYW5Ac3R1ZGVudC51dG0ubXkiLCJub3R1IjoiaSU5STRUNVUKVFRkFLRV9UT0tFTl90T19TSUdoQVRVUkVFTk9FRVhQSVZJIn0=",
4   "user": {
5     "id": 1,
6     "name": "Muhammad Aiman Hakimi",
7     "email": "aiman@student.utm.my",
8     "password": "password123",
9     "password_hash": "password123",
10    "role": "user",
11    "created_at": "2026-06-22 11:28:30"
12  },
13   "debug_received_body": {
14     "email": "aiman@student.utm.my' -- ",
15     "password": "blablabla"
16  },
17   "debug_sql": "SELECT * FROM users WHERE email = 'aiman@student.utm.my' -- ' AND password = 'blablabla' LIMIT 1"
18 }
```

Test 8: Try to update protected fields

The screenshot shows a REST client interface with a PUT request to `http://localhost:8080/api/persons/1`. The request body is a JSON object with the following content:

```
1 {
2   "name": "Ali Updated",
3   "age": 22,
4   "height": 1.70,
5   "weight": 65,
6   "user_id": 2,
7   "role": "admin",
8   "bmi": 10,
9   "category": "Normal"
10 }
```

The response status is **200 OK** with a size of **769 Bytes** and a time of **57 ms**. The response body is a JSON object with the following content:

```
1 {
2   "message": "BMI record updated. This route allows unsafe field
3     updates.",
4   "person": {
5     "id": 1,
6     "user_id": 2,
7     "name": "Ali Updated",
8     "age": 22,
9     "height": "1.70",
10    "weight": "65.00",
11    "bmi": "10.00",
12    "category": "Normal",
13    "notes": "Student sample BMI record",
14    "created_at": "2026-06-22 11:28:30"
15  },
16  "debug_received_body": {
17    "name": "Ali Updated",
18    "age": 22,
19    "height": 1.7,
20    "weight": 65,
21    "user_id": 2,
22    "role": "admin",
23    "bmi": 10,
24    "category": "Normal"
25  },
26  "debug_sql": "UPDATE persons SET user_id = 2, name = 'Ali Updated'
    , age = 22, height = 1.7, weight = 65, bmi = 10, category =
    'Normal' WHERE id = 1"
27 }
```

3. Weaknesses found

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
XSS payload executes	Frontend	XSS	Browser script execution
SQL Injection input affects query	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or role	API / Backend	Mass assignment	Privilege or ownership manipulation

Discussion Questions

1. Which weakness is the most dangerous? Why?
 - SQL Injection is among the most dangerous because it can allow attackers to bypass authentication, read, modify, or delete database data if queries are not protected.
2. Which weakness is caused by trusting the frontend?
 - Missing backend validation because attackers can bypass frontend validation using Postman or Thunder Client.
3. Which weakness is caused by missing backend validation?
 - Invalid BMI values such as negative weight, zero height, and empty names.
4. Which weakness is caused by missing authorization?
 - Broken Object Level Authorization and Broken Function Level Authorization. These allow users to access data or functions they should not have permission to use.
5. Which weakness can expose private data?
 - Sensitive Data Exposure, such as returning password_hash or allowing users to view another user's BMI records.
6. Which weakness can affect the browser?
 - Cross-Site Scripting (XSS), which can execute malicious JavaScript in the user's browser.
7. Which weakness can affect the database?
 - SQL Injection, because attackers may manipulate SQL queries to read, modify, or delete data.
8. Which weakness can be prevented by JWT?
 - JWT helps verify a user's identity (authentication) and supports protecting routes from unauthenticated access.

9. Which weakness cannot be solved by JWT alone?

- SQL Injection
- XSS
- Backend validation issues
- Owner-based access control
- RBAC (roles still need to be checked after verifying the token)

10. Which fixes must be implemented in the backend?

- Input validation
- BMI calculation
- Password hashing
- Prepared statements
- JWT verification
- RBAC checks
- Owner-based access control
- Safe API responses
- Preventing mass assignment